Passo a passo da GS — RotaSeca

Prazo final: quinta-feira, 19/11/2026, às 23h55 (20/11 é feriado e não tem prorrogação). Só **um integrante** envia cada disciplina, no Teams ou no Portal do Aluno.

Integrante	RM
Christian Schunck de Almeida	RM563850
Guilherme Vilela Perez	RM564422
Gustavo Panham Dourado	RM563904
Paulo Cesar de Govea Junior	RM566034
Thomas Jeferson Santana Wang	RM565104

Divisão sugerida (todos precisam saber explicar o projeto inteiro)

Quem	Responsável por	Prazo sugerido
Paulo	GitHub, Oracle, deploy no Render (passos 2 a 6) e conversa com o morador (passo 7)	até 12/11
Christian	Diagrama no Oracle Data Modeler (passo 4) e revisão do PDF de Database	até 12/11
Guilherme	Planning Poker (passo 8) e revisão do PDF de Agile	até 13/11
Gustavo	Revisão do notebook de Data Science e do módulo de Dynamic Programming	até 13/11
Thomas	Edição e publicação do vídeo (passo 9)	até 16/11
Todos	Gravar a própria narração, completar o `prompts.md` e ensaiar as explicações (passos 10 e 11)	até 17/11
Um integrante	Envio de todas as disciplinas (passo 12)	**até 18/11** (um dia de folga)

---

1. Preparar o computador (uma vez)

- **Java JDK 21** e **IntelliJ IDEA** (Community serve).
- **Oracle SQL Developer** (já tem o Data Modeler dentro).
- **Python 3.10+**. Para regerar os PDFs:

  ```
  pip install playwright pypdf markdown
  playwright install chromium
  ```

- **Git** e uma conta no **GitHub** e no **Render** (pode entrar com o GitHub).

2. Completar os dados que faltam

Abra `_docs_src/equipe.json` e preencha:

- `"turma"`: a turma de vocês (ex.: `"2ESPA"`);
- `"link_github"`: depois do passo 3;
- `"link_deploy"`: depois do passo 6;
- `"link_pitch"`: depois do passo 9;
- `"escuta"`: depois do passo 7.

Sempre que mudar algo, rode na pasta `_docs_src`:

```
python build_all.py --so-pdfs
```

Isso regera os PDFs de Agile, Database e DDD com os dados novos.

3. Subir o projeto no GitHub (até 10/11)

1. No GitHub: **New repository** → nome `rotaseca` → **Public** (ou privado, adicionando os professores como colaboradores).
2. Na pasta `rotaseca` (descompactada do ZIP):

   ```
   git init
   git add .
   git commit -m "RotaSeca - Global Solution 2026.2"
   git branch -M main
   git remote add origin https://github.com/SEU-USUARIO/rotaseca.git
   git push -u origin main
   ```

3. Em *Settings* → *Collaborators*, adicionem os outros 4 integrantes.
4. Coloquem o link em `equipe.json` e no `README.md` (linha "Link do repositório").

> **Atenção (DDD):** commits **depois de 19/11** zeram a parte técnica de DDD. Façam o último `git push` até 19/11 e não mexam mais no repositório depois.

4. Criar as tabelas no Oracle da FIAP (até 11/11)

1. No SQL Developer, crie uma conexão: **Host** `oracle.fiap.com.br` · **Porta** `1521` · **SID** `ORCL` · **Usuário** `rm566034` (o RM de quem vai fazer o deploy) · **Senha**: a senha do Oracle que a FIAP passou (ou a que vocês já trocaram).
2. Abra `database/01_DDL_rotaseca.sql` e execute com **F5** (*Run Script*). Depois rode o `02_DML_rotaseca.sql` e o `03_DQL_rotaseca.sql`.
3. Confira se as 3 consultas do DQL retornam linhas. Se retornarem, o banco está pronto.

5. Diagrama no Oracle Data Modeler (obrigatório em Database Design)

1. SQL Developer → **File → Data Modeler → Import → DDL File** → escolha `database/01_DDL_rotaseca.sql` → banco **Oracle Database 19c** → Next até concluir.
2. Arrume as tabelas na tela (*botão direito → Auto Layout* ajuda) e confira as 8 ligações (FKs).
3. **File → Data Modeler → Print Diagram → To Image File** → salve como `modelo_logico.png`.
4. Substitua o arquivo `_docs_src/img/modelo_logico.png` por esse e rode `python build_all.py --so-pdfs`. O PDF de Database passa a ter o diagrama do Data Modeler (o de DDD também usa essa imagem como DER).

6. Rodar a API e fazer o deploy (até 12/11)

Local:

1. No IntelliJ: *Open* → pasta `java-api`. Aguarde o Maven baixar as dependências.
2. Rode `RotaSecaApplication`. Abra `http://localhost:8080` (página) e `http://localhost:8080/swagger-ui.html`.
3. No terminal do IntelliJ: `mvn test`. Os testes devem passar. Se algum falhar, copiem a mensagem de erro e me mandem.

Render:

1. render.com → **New → Web Service** → conecte o repositório `rotaseca`.
2. **Root Directory:** `java-api` · **Runtime:** Docker · **Instance type:** Free.
3. Em **Environment Variables**:
   - `SPRING_PROFILES_ACTIVE` = `prod`
   - `DB_USER` = `rm566034`
   - `DB_PASSWORD` = senha do Oracle
4. **Create Web Service** e aguarde o build (5 a 10 minutos).
5. Teste `https://SEU-SERVICO.onrender.com/api`. Deve aparecer `"status": "UP"`. Teste também `/api/bairros`, que deve listar os 5 bairros vindos do Oracle.
6. Coloque o link em `equipe.json` (`link_deploy`) e rode `python build_all.py --so-pdfs`.

> O plano grátis "dorme" depois de 15 minutos sem acesso. Abram o link uns minutos antes de enviar e antes de qualquer apresentação.

7. Conversar com uma pessoa real do bairro (15 minutos)

O enunciado exige "conversar com alguém que convive com o problema". O Paulo mora no bairro, então basta conversar com um vizinho, parente ou comerciante. Peçam permissão, não gravem sem autorização e **não usem o nome verdadeiro no documento** se a pessoa preferir (escrevam "Sra. M., 70 anos, moradora há 30 anos", por exemplo).

Perguntas:

1. Há quanto tempo mora aqui? Como você se desloca no dia a dia (a pé, ônibus, carro)?
2. Quais ruas ou lugares do bairro mais alagam quando chove forte?
3. Conte a última vez que a chuva te pegou na rua. O que aconteceu?
4. Como você fica sabendo hoje que uma rua alagou?
5. O que mais atrapalha nesses dias?
6. Se recebesse um aviso no celular, como gostaria que fosse (WhatsApp, áudio, SMS)?
7. Mostre a tela "Agora" do protótipo: o que você entende? O que mudaria?

Depois preencham no `equipe.json`:

```
"escuta": {
  "pessoa": "Sra. M., 70 anos, moradora ha 30 anos",
  "data_local": "08/11/2026, Parque Novo Mundo",
  "faz_hoje": "...",
  "atrapalha": "...",
  "frase": "..."
}
```

Rodem `python build_all.py --so-pdfs`. O quadro laranja do PDF de Agile fica azul e preenchido. Se as respostas mudarem alguma ideia (por exemplo, a pessoa prefere SMS), vale ajustar as personas ou o backlog.

8. Planning Poker de verdade (10 minutos)

Os votos que estão no PDF de Agile são uma sugestão. Façam a sessão (pode ser pelo planningpoker.live ou com cartas no papel) para as histórias US04, US05, US06 e US12:

1. Cada um vota ao mesmo tempo (1, 2, 3, 5, 8, 13, 21).
2. Quem deu o menor e o maior voto explica o porquê.
3. Votem de novo até haver consenso.

Atualizem os votos na lista `POKER` dentro de `_docs_src/doc_agile.py` (a ordem dos votos é a dos nomes em ordem alfabética: Christian, Guilherme, Gustavo, Paulo, Thomas) e, se a estimativa final mudar, o `SP` da história na lista `BACKLOG`. Depois rodem `python build_all.py --so-pdfs`.

9. Gravar e publicar o vídeo (até 16/11)

Sigam o **`ROTEIRO_VIDEO.md`**: falas prontas para cada integrante, o que mostrar em cada cena e o checklist de publicação. Resumo:

- máximo de 3 minutos, narração dos integrantes, YouTube **Não listado**;
- teste o link em janela anônima;
- link no `equipe.json` (`link_pitch`) e PDFs regerados;
- o mesmo link vai para **Agile** (dentro do PDF) e para **Testing** (na tarefa).

10. Completar o `prompts.md`

Cada integrante acrescenta os prompts que usou com IA e confere se sabe explicar cada parte. O que mais cai em pergunta:

- **DDD:** como uma leitura vira alerta (`AlertaBO.avaliarNivel`) e por que o mesmo nível não gera alerta repetido.
- **Data Science:** por que a divisão treino/teste é por data, e por que a lâmina d'água não entra na regressão logística (vazamento de dados).
- **Dynamic Programming:** a recorrência da mochila (memoização x tabulação) e a do Floyd-Warshall.
- **Database:** cardinalidades e o que cada uma das 3 consultas faz.

11. Revisão final (17/11)

- ☐ PDFs abertos e conferidos: capa com os 5 nomes e RMs, turma, links corretos.
- ☐ Diagrama do Data Modeler no PDF de Database.
- ☐ Deploy respondendo e GitHub atualizado (último push até 19/11).
- ☐ Notebook abrindo com os gráficos.
- ☐ `python main.py --demo` rodando na pasta `dynamic-programming`.
- ☐ Gerar de novo os ZIPs com os arquivos atualizados (tabela abaixo).

12. O que enviar em cada tarefa (até 18/11)

Disciplina	O que enviar	Formato exigido
Agile	`agile/RotaSeca_Agile.pdf`	**um único PDF** (mais de um PDF ou link externo = −10 pontos), com o link do vídeo
Database Design	PDF + `01_DDL`, `02_DML`, `03_DQL` (`.sql`)	PDF com itens 1 e 2; scripts em arquivos `.sql`; sem links de diagramas em nuvem
DDD Java	ZIP com `RotaSeca_DDD_Java.pdf` + `LINKS.txt` (GitHub e deploy)	`.zip` pelo **Portal do Aluno**
Data Science	ZIP com o `.ipynb` executado, o CSV e o `integrantes.txt`	`.zip`, um envio por equipe
Dynamic Programming	ZIP com o código-fonte e o `README.md`	`.zip` no Teams
Testing	link do vídeo no YouTube	link
AR-VR / Network	*fora deste pacote*	—

Para montar os ZIPs atualizados (no Windows: botão direito → *Compactar para arquivo ZIP*):

- **Database:** `database/RotaSeca_DatabaseDesign.pdf` + os três `.sql`.

- **DDD:** `java-api/RotaSeca_DDD_Java.pdf` + `java-api/LINKS.txt` (atualize os links nele).
- **Data Science:** `RotaSeca_DataScience.ipynb`, `rotaseca_chuvas_alagamentos.csv`, `integrantes.txt`, `FONTE_DOS_DADOS.txt`, `gerar_dados.py`.
- **Dynamic Programming:** `README.md`, `main.py` e as pastas `rotaseca_dp` e `tests`.